

TASK 13: ASSIGNMENT NO. 13

Name of Student : Anuja Vilas Wankhade

Domain: Artificial intelligence and machine Learning (AIML)

College Name: Manav School of Polytechnic AKOLA

Branch : Computer Engineering

GitHub Repository Link : <https://github.com/anujaw193-star/python-project-collection>

Q1. (Basic Array Creation)

PROGRAM CODE

```
import numpy as np
# a) Create a 1D array
arr1 = np.array([10, 20, 30, 40, 50])
# b) Create a 2D array (3x3)
arr2 = np.array([
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
])
# Print 1D array details
print("1D Array:")
print(arr1)
print("Shape:", arr1.shape)
print("Data Type:", arr1.dtype)
# Print 2D array details
print("\n2D Array:")
print(arr2)
print("Shape:", arr2.shape)
print("Data Type:", arr2.dtype)
```

OUTPUT:

```

PS C:\Users\anuja\OneDrive\Desktop\ITR AI ML> python Task13.py
1D Array:
[10 20 30 40 50]
Shape: (5,)
Data Type: int64

2D Array:
[[1 2 3]
 [4 5 6]
 [7 8 9]]
Shape: (3, 3)
Data Type: int64
PS C:\Users\anuja\OneDrive\Desktop\ITR AI ML>

```

Q2. (np.zeros() and np.ones())

PROGRAM CODE :

```

import numpy as np
# a) 1D array of 8 zeros
arr1 = np.zeros(8)
# b) 2D array of shape (4, 4) filled with ones
arr2 = np.ones((4, 4))
# c) 3x3 matrix of zeros
arr3 = np.zeros((3, 3))
# Print arrays with labels
print("a) 1D Array of 8 Zeros:")
print(arr1)
print("\nb) 4x4 Array of Ones:")
print(arr2)
print("\nc) 3x3 Matrix of Zeros:")
print(arr3)

```

OUTPUT :

```

Data Type: int64
PS C:\Users\anuja\OneDrive\Desktop\ITR AI ML> python Task13.py
a) 1D Array of 8 Zeros:
[0. 0. 0. 0. 0. 0. 0. 0.]

b) 4x4 Array of Ones:
[[1. 1. 1. 1.]
 [1. 1. 1. 1.]
 [1. 1. 1. 1.]
 [1. 1. 1. 1.]]

c) 3x3 Matrix of Zeros:
[[0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]]
PS C:\Users\anuja\OneDrive\Desktop\ITR AI ML>

```

Q3. (np.arange())

PROGRAM CODE:

```
import numpy as np
# a) Numbers from 0 to 20 (step 1)
a = np.arange(0, 21, 1)
# b) Even numbers from 10 to 50
b = np.arange(10, 51, 2)
# c) Numbers from 5 to 100 with step of 5
c = np.arange(5, 101, 5)
print("a) Numbers from 0 to 20:")
print(a)
print("\nb) Even numbers from 10 to 50:")
print(b)
print("\nc) Numbers from 5 to 100 with step of 5:")
print(c)
```

OUTPUT:

```
PS C:\Users\anuja\OneDrive\Desktop\ITR AIML> python Task13.py
a) Numbers from 0 to 20:
[ 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20]

b) Even numbers from 10 to 50:
[10 12 14 16 18 20 22 24 26 28 30 32 34 36 38 40 42 44 46 48 50]

c) Numbers from 5 to 100 with step of 5:
[ 5 10 15 20 25 30 35 40 45 50 55 60 65 70 75 80 85 90
 95 100]
PS C:\Users\anuja\OneDrive\Desktop\ITR AIML> █
```

Q4. (np.linspace())

PROGRAM CODE:

```
import numpy as np
# a) 10 equally spaced numbers between 0 and 5
arr1 = np.linspace(0, 5, 10)
# b) 15 equally spaced numbers between -10 and 10
arr2 = np.linspace(-10, 10, 15)
# Print arrays and their lengths
print("a) 10 equally spaced numbers between 0 and 5:")
print(arr1)
print("Length:", len(arr1))
print("\nb) 15 equally spaced numbers between -10 and 10:")
print(arr2)
print("Length:", len(arr2))
```

OUTPUT:

```

PS C:\Users\anuja\OneDrive\Desktop\ITR AIML> Python Task13.py
a) 10 equally spaced numbers between 0 and 5:
[0.      0.55555556 1.11111111 1.66666667 2.22222222 2.77777778
 3.33333333 3.88888889 4.44444444 5.      ]
Length: 10

b) 15 equally spaced numbers between -10 and 10:
[-10.      -8.57142857 -7.14285714 -5.71428571 -4.28571429
 -2.85714286 -1.42857143  0.      1.42857143  2.85714286
  4.28571429  5.71428571  7.14285714  8.57142857 10.      ]
Length: 15
PS C:\Users\anuja\OneDrive\Desktop\ITR AIML> 

```

Q5. (Random Arrays)

PROGRAM CODE:

```

import numpy as np
# a) 1D array of 10 random numbers between 0 and 1
arr1 = np.random.rand(10)
# b) 3x3 matrix of random numbers from standard normal distribution
arr2 = np.random.randn(3, 3)
# c) 4x5 array of random integers between 10 and 50
arr3 = np.random.randint(10, 51, size=(4, 5))
# Print results
print("a) 1D Array of 10 Random Numbers (0 to 1):")
print(arr1)
print("\nb) 3x3 Matrix of Random Numbers (Standard Normal Distribution):")
print(arr2)
print("\nc) 4x5 Array of Random Integers (10 to 50):")
print(arr3)

```

OUTPUT:

```

PS C:\Users\anuja\OneDrive\Desktop\ITR AIML> Python Task13.py
a) 1D Array of 10 Random Numbers (0 to 1):
[0.47144505 0.11258374 0.55549108 0.55998665 0.08749054 0.03128345
 0.97596465 0.80579637 0.36579509 0.23451558]

b) 3x3 Matrix of Random Numbers (Standard Normal Distribution):
[[ 0.80077918 -0.0135034 -0.46841568]
 [-0.65424311  1.23037645 -1.00811832]
 [-0.51910822 -0.27070231 -0.50328596]]

c) 4x5 Array of Random Integers (10 to 50):
[[35 18 15 39 42]
 [23 29 30 23 12]
 [14 34 22 23 11]
 [37 50 28 12 27]]
PS C:\Users\anuja\OneDrive\Desktop\ITR AIML> 

```

Q6. (Vectors and Basic Operations)

PROGRAM CODE :

```
import numpy as np
# Create vectors
v1 = np.array([2, 4, 6, 8])
v2 = np.array([1, 3, 5, 7])
# Operations
addition = v1 + v2
subtraction = v1 - v2
multiplication = v1 * v2
dot_product = np.dot(v1, v2)
# Print results
print("Vector v1:", v1)
print("Vector v2:", v2)
print("\nAddition:")
print(addition)
print("\nSubtraction:")
print(subtraction)
print("\nElement-wise Multiplication:")
print(multiplication)
print("\nDot Product:")
print(dot_product)
```

OUTPUT :

```
PS C:\Users\anuja\OneDrive\Desktop\ITR AIML> Python Task13.py
Vector v1: [2 4 6 8]
Vector v2: [1 3 5 7]

Addition:
[ 3  7 11 15]

Subtraction:
[1 1 1 1]

Element-wise Multiplication:
[ 2 12 30 56]

Dot Product:
100
PS C:\Users\anuja\OneDrive\Desktop\ITR AIML> |
```

Q7. (Matrices and Operations)

PROGRAM CODE :

```
import numpy as np
# Create matrices
A = np.array([[1, 2, 3],
```

```

        [4, 5, 6],
        [7, 8, 9]])
B = np.array([[9, 8, 7],
              [6, 5, 4],
              [3, 2, 1]])
# Matrix Addition
addition = A + B
# Matrix Multiplication
matrix_multiplication = A @ B
# or np.dot(A, B)
# Element-wise Multiplication
elementwise_multiplication = A * B
# Print results
print("Matrix A:")
print(A)
print("\nMatrix B:")
print(B)
print("\nMatrix Addition (A + B):")
print(addition)
print("\nMatrix Multiplication (A @ B):")
print(matrix_multiplication)
print("\nElement-wise Multiplication (A * B):")
print(elementwise_multiplication)

```

OUTPUT :

```

PS C:\Users\anuja\OneDrive\Desktop\ITR AIML> Python Task13.py
Matrix A:
[[1 2 3]
 [4 5 6]
 [7 8 9]]

Matrix B:
[[9 8 7]
 [6 5 4]
 [3 2 1]]

Matrix Addition (A + B):
[[10 10 10]
 [10 10 10]
 [10 10 10]]

Matrix Multiplication (A @ B):
[[ 30  24  18]
 [ 84  69  54]
 [138 114  90]]

Element-wise Multiplication (A * B):
[[ 9 16 21]
 [24 25 24]
 [21 16  9]]
PS C:\Users\anuja\OneDrive\Desktop\ITR AIML>

```

Q8. (Properties of Arrays)

PROGRAM CODE :

```

import numpy as np
# Create a 4x4 matrix of random integers between 1 and 100
arr = np.random.randint(1, 101, size=(4, 4))
# Print the matrix
print("4x4 Random Integer Matrix:")
print(arr)
# Properties
print("\nShape:", arr.shape)
print("Dimension (ndim):", arr.ndim)
print("Total Number of Elements (size):", arr.size)
print("Data Type:", arr.dtype)
print("Minimum Value:", arr.min())
print("Maximum Value:", arr.max())

```

OUTPUT:

```

PS C:\Users\anuja\OneDrive\Desktop\ITR AML> python Task13.py
4x4 Random Integer Matrix:
[[60 58 17 32]
 [38 36 21 43]
 [ 4 53 60 58]
 [ 1 12  3 15]]

Shape: (4, 4)
Dimension (ndim): 2
Total Number of Elements (size): 16
Data Type: int32
Minimum Value: 1
Maximum Value: 60
PS C:\Users\anuja\OneDrive\Desktop\ITR AML>

```

Q9. (Combined – Random + Reshape + Statistics)

PROGRAM CODE:

```

import numpy as np
# Generate a 1D array of 20 random integers between 1 and 50
arr = np.random.randint(1, 51, 20)
# Reshape into a 4x5 matrix
matrix = arr.reshape(4, 5)
# Print the matrix
print("4x5 Matrix:")
print(matrix)
# Statistics
print("\nSum of all elements:", np.sum(matrix))
print("Mean of all elements:", np.mean(matrix))
print("Standard Deviation:", np.std(matrix))
# Maximum value in each row
print("\nMaximum Value in Each Row:")
print(np.max(matrix, axis=1))

```

OUTPUT:

```
PS C:\Users\anuja\OneDrive\Desktop\ITR AIML> Python Task13.py
4x5 Matrix:
[[37 29 15 17 23]
 [11 48 10 35 31]
 [15 50 7 45 17]
 [2 21 2 38 20]]

Sum of all elements: 473
Mean of all elements: 23.65
Standard Deviation: 14.419691397529977

Maximum Value in Each Row:
[37 48 50 38]
PS C:\Users\anuja\OneDrive\Desktop\ITR AIML>
```

Q10 (Mini Project – NumPy Application)

PROGRAM CODE:

```
import numpy as np
try:
    # Ask the user for the number of random numbers
    n = int(input("Enter how many numbers you want to generate: "))
    if n <= 0:
        print("Please enter a positive number.")
    else:
        # Generate random numbers between 10 and 100
        arr = np.random.randint(10, 101, n)
        # Display the array
        print("\nGenerated Array:")
        print(arr)
        # Statistics
        print("\n--- Statistics ---")
        print("Mean:", np.mean(arr))
        print("Median:", np.median(arr))
        print("Standard Deviation:", np.std(arr))
        print("Minimum Value:", np.min(arr))
        print("Maximum Value:", np.max(arr))
        # Try to reshape into a 2D array
        print("\n--- Reshape Operation ---")
        rows = int(np.sqrt(n))
        if rows > 1 and n % rows == 0:
            cols = n // rows
            matrix = arr.reshape(rows, cols)
            print(f"\nReshaped Matrix ({rows}x{cols}):")
            print(matrix)
            print("\nRow-wise Sum:")
            print(np.sum(matrix, axis=1))
        else:
```



```
        print("Reshaping into a proper 2D matrix is not possible.")

except ValueError:
    print("Invalid input! Please enter an integer.")
except Exception as e:
    print("An error occurred:", e)
finally:
    print("\nProgram execution completed.")
```

OUTPUT:

```
PS C:\Users\anuja\OneDrive\Desktop\ITR AIML> Python Task13.py
Enter how many numbers you want to generate: 12

Generated Array:
[17 29 60 64 90 51 98 76 67 74 65 39]

--- Statistics ---
Mean: 60.833333333333336
Median: 64.5
Standard Deviation: 22.748015786486132
Minimum Value: 17
Maximum Value: 98

--- Reshape Operation ---

Reshaped Matrix (3x4):
[[17 29 60 64]
 [90 51 98 76]
 [67 74 65 39]]

Row-wise Sum:
[170 315 245]

Program execution completed.
PS C:\Users\anuja\OneDrive\Desktop\ITR AIML>
```